

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

TRIPLA: Trierer Programmiersprache

Gegeben sei die Grammatik G:

```

E → let D in E
   | ID
   | ID ( A )
   | E AOP E
   | ( E )
   | CONST
   | ID = E
   | E ; E
   | if B then E else E
   | while B do { E }
A → E | A , E
D → ID ( V ) { E }
   | D D
V → ID | V , V
B → E | E RELOP E

```

ID: Bezeichner (fangen mit Buchstaben an und bestehen aus Buchstaben und Zahlen)

CONST: ganze Zahlen und null

AOP: arithm. Operatoren (+, -, *, /)

RELOP: Vergleichsoperatoren (==, !=, <, >)

Menge aller gültigen Programme: $LG(E) = \{w \mid E \rightarrow^* w\}$

Beispiel:

```

let f1(b) {
           if (b==0) then 0 else f1(b-1)
        }

        f2(a,b) {
           if (a>b) then f1(a) else f1(b)
        }
in f1(10); f2(10,20)

```

Im **Projekt** werden Sie eine Abstrakte Maschine und einen Compiler für TRIPLA implementieren!!!



TRIPLA: Trierer Programmiersprache

Beispiel:

DEKLARATIONEN

```
let f1(b) {
    if (b==0) then 0 else f1(b-1)
}

f2(a,b) {
    if (a>b) then f1(a) else f1(b);
    let g(c) {
        a*b*c
    }
    in g(a*b)
}
```

AUFRUFE

```
in f1(10);
f2(10, let max(a,b) {
    if (a>b) then a else b
}
in max(20,30) )
```

Parameterlose Methoden sind nicht erlaubt!

Es existiert immer ein else-Zweig

Rückgabewert der Funktion, kein "return"

Verschachtelungen von Funktionen

Zugriff auf Variablen umschließender Funktionen

Rückgabewert des let-in-Ausdrucks

TRIPLA: Trierer Programmiersprache

- Weitere Besonderheiten/Einschränkungen
 - keine lokalen Variablen
 - durch Funktionsparameter realisierbar
 - Zugriff auf sämtliche übergeordneten Funktionsparameter
 - Ausnahme: Parameter können überdeckt werden!
 - Nur ganze Zahlen (Integer)
 - Der Wert des letzten Ausdrucks ist automatisch der Rückgabewert einer Funktion bzw. eines let-in-Ausdrucks
 - Der Wert von `while (x>0) { ... }` ist null, falls schon zu Beginn $x \leq 0$



TRAM

Trierer Abstrakte Maschine

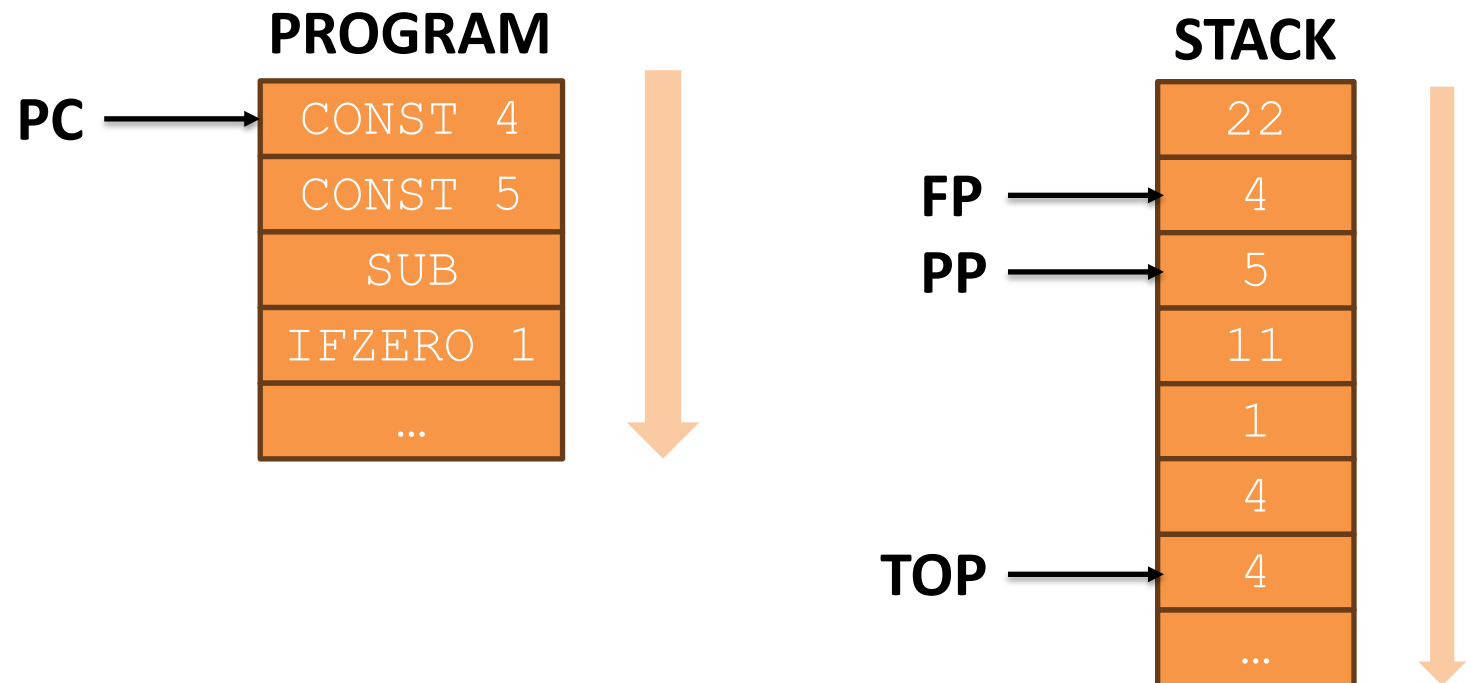


TRAM : Trierer Abstrakte Maschine

- Register:

- **PC** : zeigt auf aktuell auszuführende Instruktion (program counter)
- **PP**: zeigt auf das erste Argument (parameter pointer)
- **FP**: zeigt auf den Beginn des Kellerrahmens (frame pointer)
- **TOP**: zeigt auf die oberste, belegte Kellerzelle

```
while (PC >= 0)
    execute(program[PC]);
```



MaMa $\rightarrow$ TRAM

TRAM=(K,P,step) mit $N = \text{int} \cup \{ \text{null} \}$

- Konfigurationsmenge $K \in N \times N \times N \times N \times (N \rightarrow N)$,
- Programmspeicher $P \in N \rightarrow I$
- und der Übergangsfunktion $\text{step} \in K \rightarrow K$.

Eine Konfiguration $(\text{top}, \text{pc}, \text{pp}, \text{fp}, S) \in K$ besteht aus dem Kellerzeiger (Stackpointer) top , dem Programmzähler pc , dem Parameterzeiger (parameter pointer) pp , dem Rahmenzeiger (frame pointer) fp und dem Keller (Stack) S .

„Imperative Notation“

step(top,pc,pp,fp,S)
=(top+1,pc+1,pp,fp,S[top+1\k])

falls $P(pc)=const(k)$



const k : STACK[TOP+1] = k
 TOP = TOP + 1
 PC = PC + 1

Abstrakte Maschine: Ausdrücke

const k : $STACK[TOP+1] = k$
 $TOP = TOP + 1$
 $PC = PC + 1$

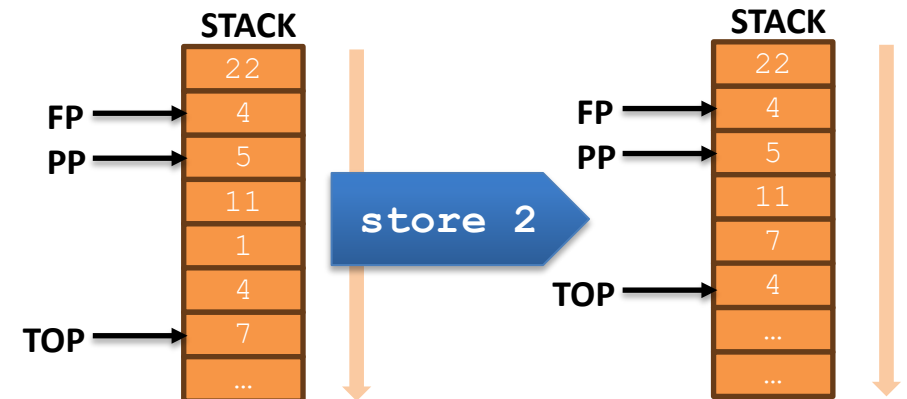
Sonderfall:
 const null

store k : $STACK[PP+k] = STACK[TOP]$
 $TOP = TOP - 1$
 $PC = PC + 1$

load k : $STACK[TOP+1] = STACK[PP+k]$
 $TOP = TOP + 1$
 $PC = PC + 1$

add: $STACK[TOP-1] = STACK[TOP-1] + STACK[TOP]$
 $TOP = TOP - 1$
 $PC = PC + 1$

sub, mul, div
 sind analog definiert



Beispiel:

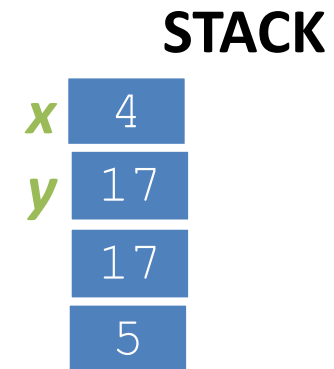
- Quellcode: $y = x * 3 + 5$
- Annahme:
 - Variable x durch Kellerzelle 0 und Variable y durch Kellerzelle 1 implementiert, sowie $PP=0$, $FP=0$ und $TOP=1$. Anfangswert von x sei 4.

```

0: load 0
1: const 3
2: mult
3: const 5
4: add
5: store 1
6: halt

```

PC	PP	FP	TOP
0	0	0	1
1	0	0	2
2	0	0	3
3	0	0	2
4	0	0	3
5	0	0	2
6	0	0	1
-1	0	0	1



Beispiel:

- Bemerkungen:
 - Die Kellerzelle 0 enthält zu Beginn bereits einen Wert für **x**
 - TOP zeigt nach Halten des Programms auf das Ergebnis **y**, wovon man i.d.R. allerdings nicht ausgehen kann. Zur Weiterverwendung von **y**, dieses zuerst mit **load 1** auf den Stack legen

```

0: load 0
1: const 3
2: mult
3: const 5
4: add
5: store 1
6: halt

```

PC	PP	FP	TOP
0	0	0	1
1	0	0	2
2	0	0	3
3	0	0	2
4	0	0	3
5	0	0	2
6	0	0	1
-1	0	0	1

STACK

x	4
y	17

Abstrakte Maschine: Vergleich

Konvention: **0** entspricht *false*, jeder Wert **ungleich 0** entspricht *true*

```
lt: if (STACK[TOP-1] < STACK[TOP])  
    STACK[TOP-1] = 1           // true  
else  
    STACK[TOP-1] = 0           // false  
TOP = TOP - 1  
PC = PC + 1
```

gt, eq, neg
sind analog definiert

Abstrakte Maschine: Kontrollfluss

goto p: PC = p

ifzero p : **if** (STACK[TOP] == 0)
 PC = p
 else
 PC = PC + 1
 TOP = TOP - 1

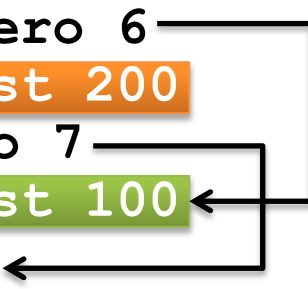
halt: PC = -1

nop: PC = PC + 1 *// no operation*

Beispiel:

- Quellcode: `x=10; if (x==0) then 100 else 200; 3`
- Annahmen:
 - Variable x durch Kellerzelle 0 implementiert, sowie PP=0, FP=0 und TOP=0

```
0: const 10
1: store 0
2: load 0
3: ifzero 6
4: const 200
5: goto 7
6: const 100
7: nop
8: const 3
9: halt
```



TRIPLA: Trierer Programmiersprache

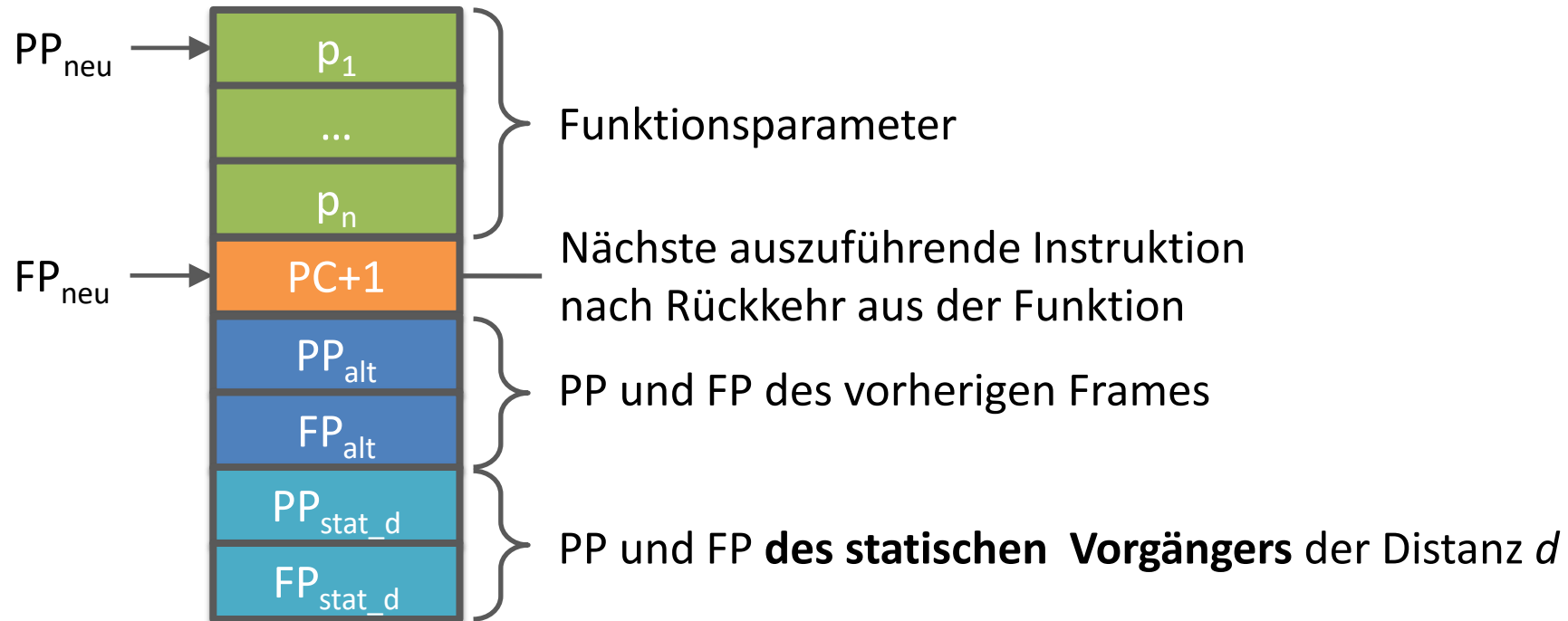
Funktionen und Parameter

- Weitere Besonderheiten/Einschränkungen
 - Zugriff auf sämtliche übergeordneten Funktionsparameter
 - **Achtung:**
 - Parameter können überdeckt werden!
 - Parameter benachbarter Funktionen sind nicht sichtbar!
 - **Analog für Sichtbarkeit von in benachbarten Funktionen deklarierten Funktionen.**

Die Maschine selbst kann auf überdeckte Parameter zugreifen.
Unsere Programmiersprache stellt allerdings kein Konstrukt bereit, mit dem man die Überdeckung umgehen könnte.

Abstrakte Maschine: Funktionen

Realisierung von Funktionen durch **Kellerrahmen** (stack frames):

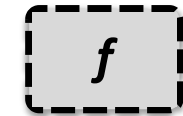
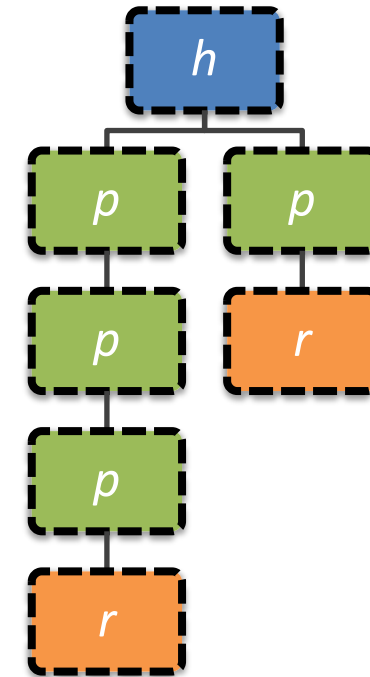


Dynamische Aufrufbäume

```

let h(i) {
  let p(a) {
    let r(b)
    | { ... }
    in
    if i > 0
    then i := i-1; p(0)
    else r(0)
  in
  i := 2;
  p;
  p
}

```



Inkarnation
einer Prozedur
bzw. Funktion
zur Laufzeit

- Es können mehrere Aufrufbäume für ein Programm existieren
- Unendliche Aufrufbäume sind möglich

Alternative Syntax

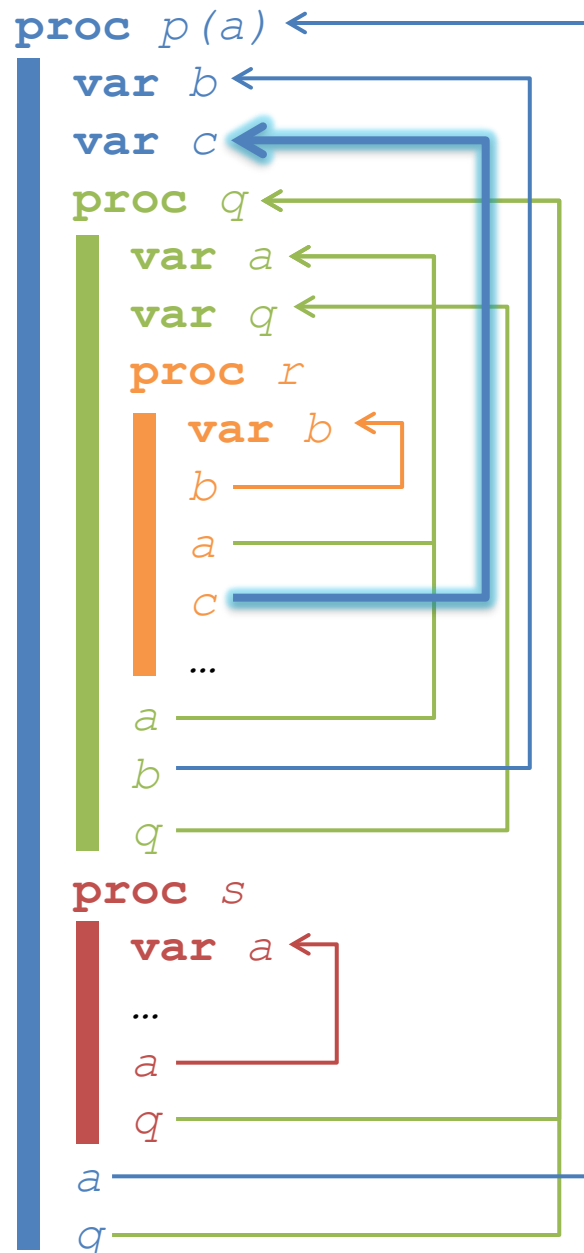
```

program h
  var i : integer;
  proc p
    proc r
      ...
      if i > 0
        then i := i-1; p
        else r
      fi
    i := 2;
    p;
    p
  
```

```

let h(i) {
  let p(a) {
    let r(b)
    { ... }
    in
      if i > 0
        then i := i-1; p(0)
        else r(0)
    in
      i := 2;
      p;
      p
  }
}

```



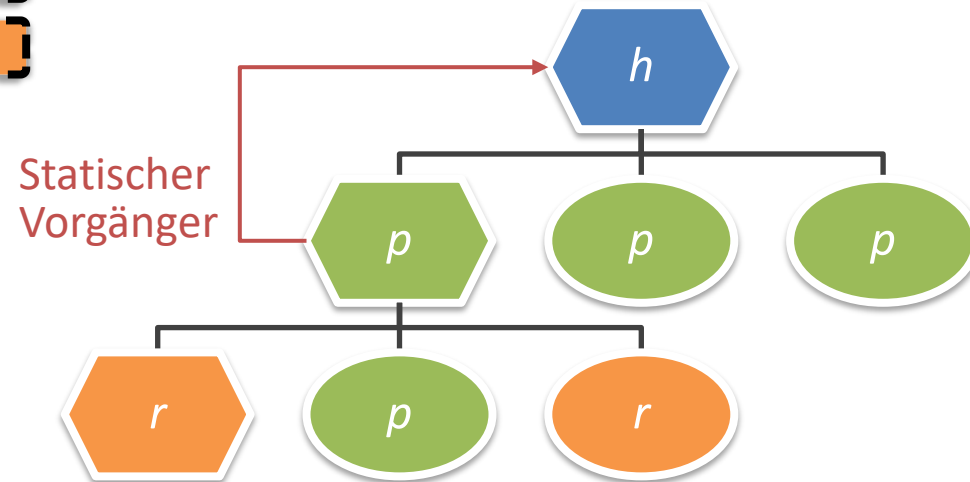
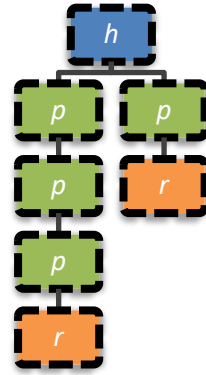
Statische Bindung

- **Definierte** (Deklaration mit `proc`, `var`) und **angewandte** (Aufruf) Vorkommen von Prozeduren und Variablen
- Analyse beruht nur auf Quelltext → statisch
- **Statische Bindung:** Zuordnung von angewandten zu definierten Vorkommen unter Berücksichtigung der Sichtbarkeit

Statischer Vorgänger im Programmtext

```

program h
  var i : integer;
  proc p
    proc r
      ...
    if i > 0
      then i := i-1; p
      else r
    fi
  i := 2;
  p;
  p
  
```



Deklaration

Aufruf

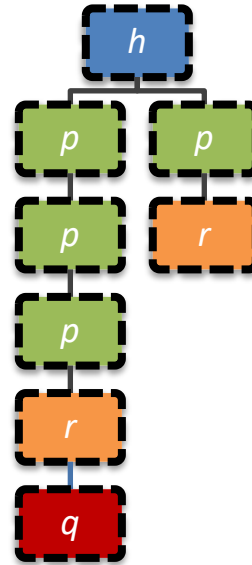
Vorkommen im
Programmtext

Beobachtung: Eine Funktion kann nur aufgerufen werden, wenn mindestens ein Aufruf ihres statischen Vorgängers aktiv ist.

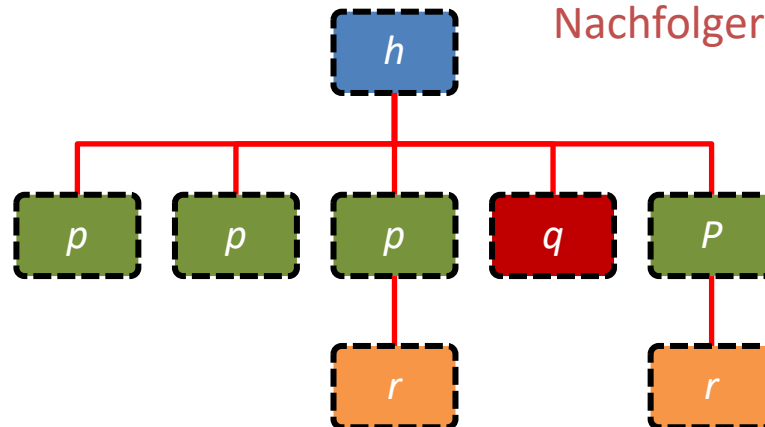
Baum der statischen Nachfolger zur Laufzeit

```

program h
  var i : integer;
  proc p
    |
    |   proc r
    |   |   q
    |   |   if i > 0
    |   |   |   then i := i-1; p
    |   |   |   else r
    |   |   fi
    |   i := 2;
    |   p;
    |   p
    |   proc q
    |   |   ...
  
```

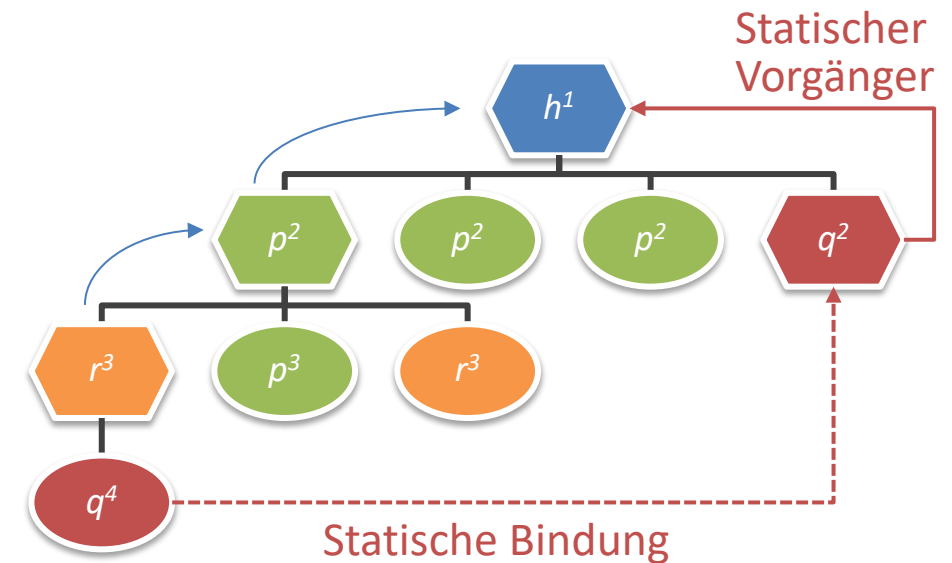


Statische
Nachfolger



Deklaration

Aufruf

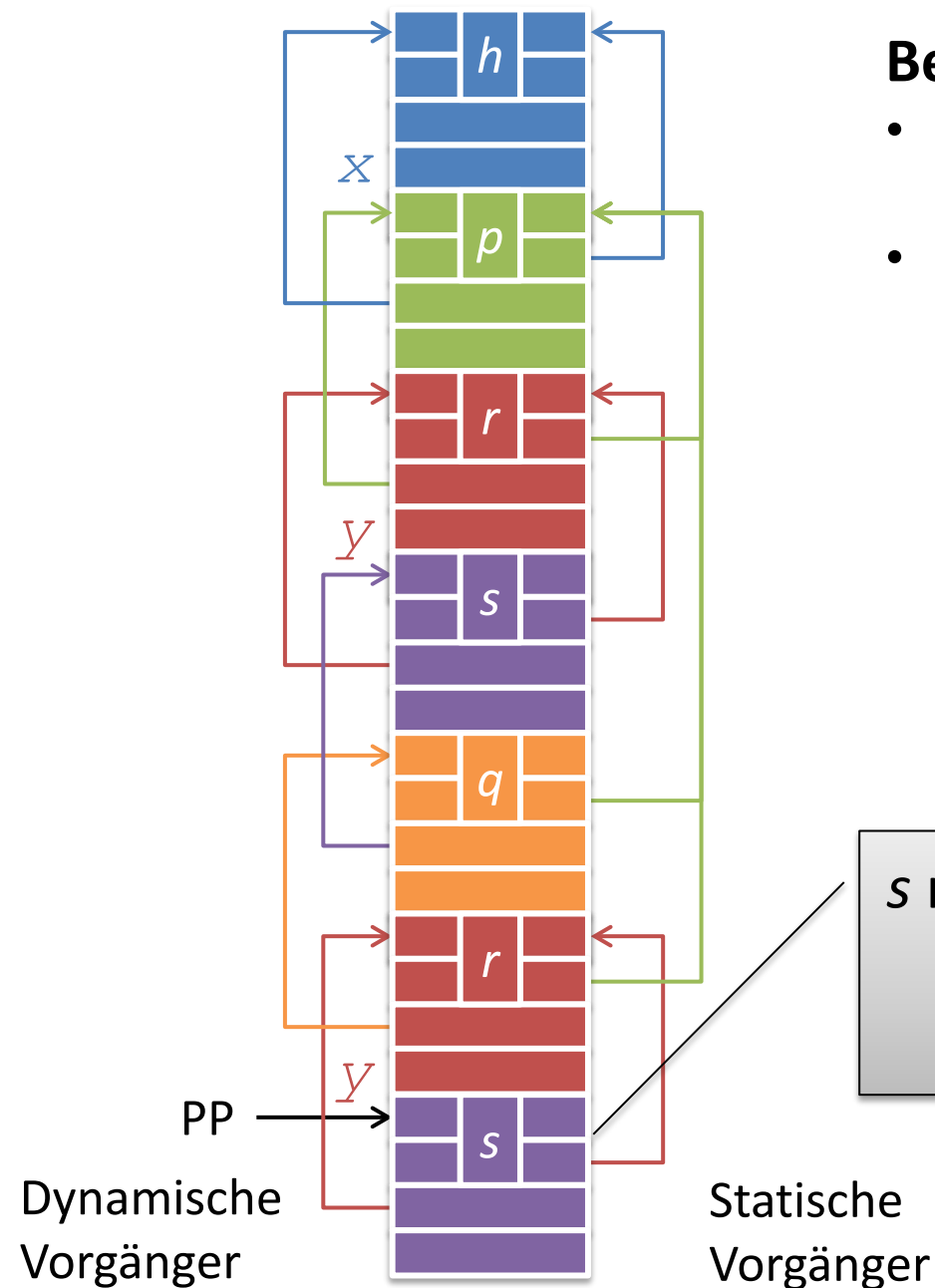


$$d = 4 - 2 = 2$$

q wird nicht in der Umgebung von r sondern von h ausgeführt!

```

program  $h^1$ 
  var  $x$ 
  proc  $p^2$ 
    ...
    proc  $q^3$ 
      ...
       $r^4$ 
      ...
      proc  $r^3$ 
        var  $y$ 
        proc  $s^4$ 
           $x+y$ 
           $q^5$ 
           $s$ 
         $r$ 
       $r$ 
     $p$ 
  
```

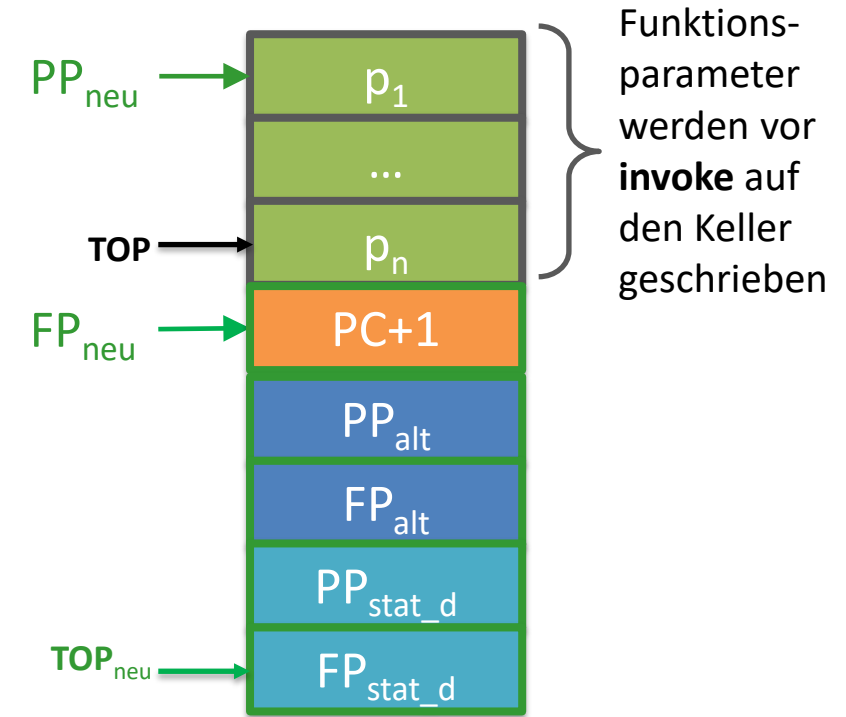
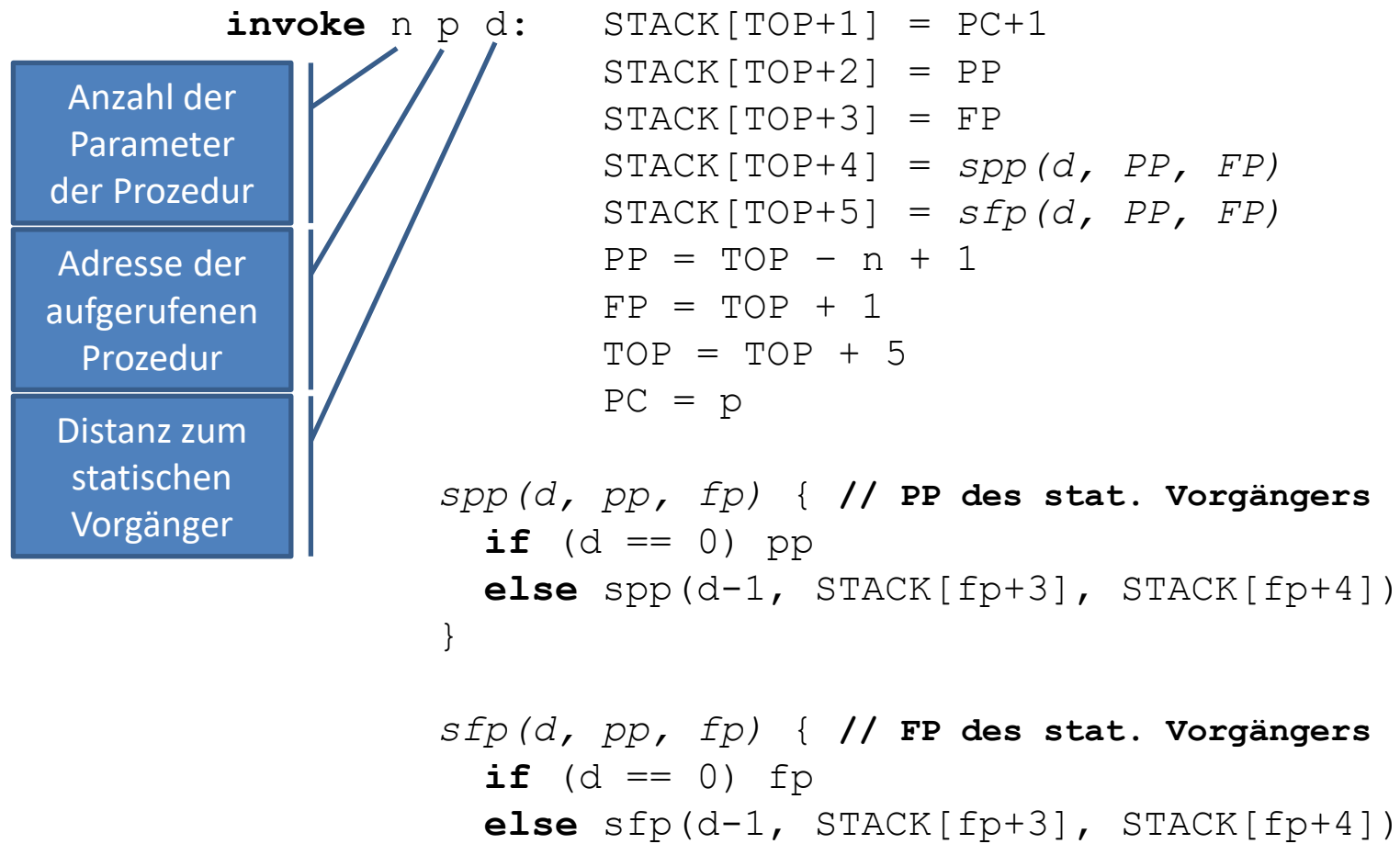


Bemerkungen:

- Die TRAM arbeitet nur mit statischen Vorgängern.
- Um den statischen Vorgänger der Distanz d für eine Funktion zu erhalten, muss man also ausgehend von deren Deklaration im Baum der statischen Vorgänger d Stufen nach oben laufen.

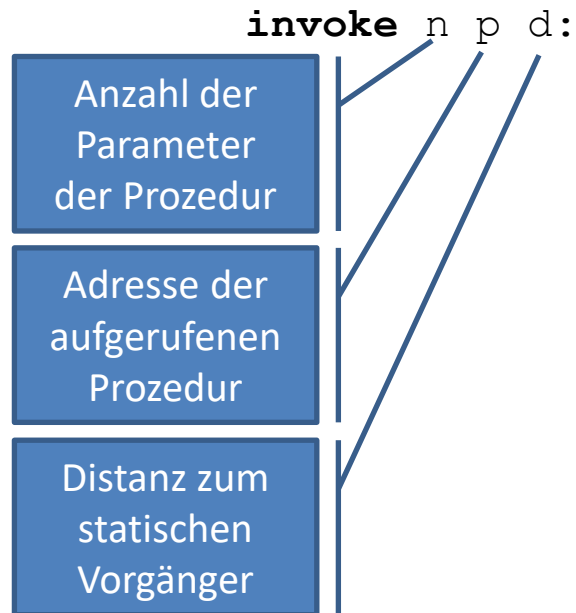
s ruft q auf. $d = 5 - 3 = 2$.
Ausführung also in Umgebung von p .

Abstrakte Maschine: Funktionsaufruf



Abstrakte Maschine

Funktionsaufruf



```

STACK[TOP+1] = PC+1
STACK[TOP+2] = PP
STACK[TOP+3] = FP
STACK[TOP+4] = spp(d, PP, FP)
STACK[TOP+5] = sfp(d, PP, FP)
PP = TOP - n + 1
FP = TOP + 1
TOP = TOP + 5
PC = p

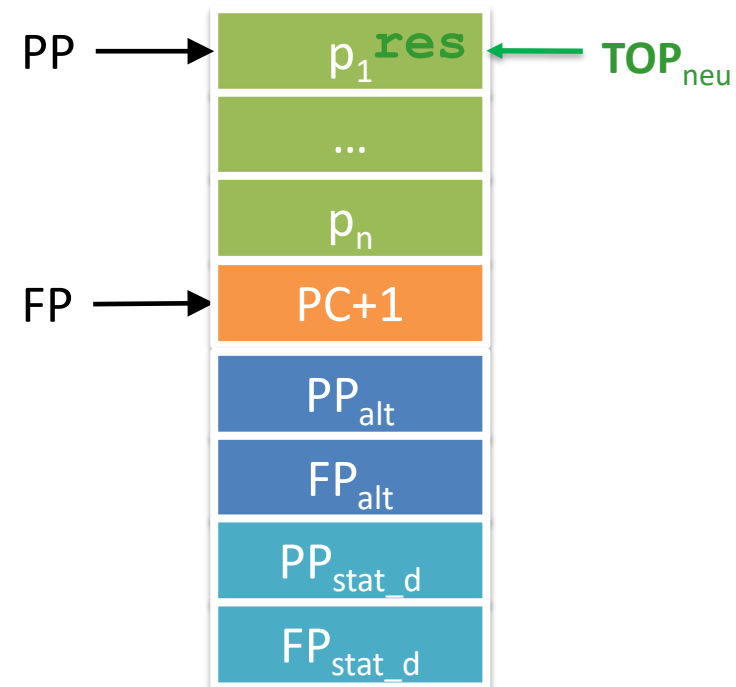
```

return :

```

res = STACK[TOP]
TOP = PP
PC = STACK[FP]
PP = STACK[FP+1]
FP = STACK[FP+2]
STACK[TOP] = res

```



Erweiterte Maschinenbefehle

Ermöglichen Zugriff auf Parameter der statischen Vorgänger:

store *k* *d*: $\text{STACK}[\text{spp}(\mathbf{d}, \mathbf{PP}, \mathbf{FP}) + k] = \text{STACK}[\text{TOP}]$
 $\text{TOP} = \text{TOP} - 1$
 $\text{PC} = \text{PC} + 1$

load *k* *d*: $\text{STACK}[\text{TOP} + 1] = \text{STACK}[\text{spp}(\mathbf{d}, \mathbf{PP}, \mathbf{FP}) + k]$
 $\text{TOP} = \text{TOP} + 1$
 $\text{PC} = \text{PC} + 1$

Die erweiterten Maschinenbefehle
ersetzen **store** *k* und **load** *k*

Beispiel:

- Quellcode: **let** `square(x) { x*x }` **in** `square(10)`
- Annahmen:
 - Das Argument von *square* wird durch Kellerzelle 0 repräsentiert, sowie PP=0, FP=0 und TOP=-1

```
0: const 10
1: invoke 1 3 0
2: halt ←
3: load 0 0 ←
4: load 0 0
5: mult
6: return ←
```

```
graph TD
    1[1: invoke 1 3 0] --> 3[3: load 0 0]
    3 --> 2[2: halt]
    6[6: return] --> 2
```

Beispiel:

Annahmen: Die Argumente von *wrapper* werden durch die Kellerzellen 0 und 1 repräsentiert, sowie PP=0, FP=0 und TOP=-1

```
let wrapper(number, threshold) {
  let square(x) {
    if (x*x > threshold)
    then x
    else x*x
  }
  in square(number)
}
in wrapper(4, 10)
```

```
0:  const 4
1:  const 10
2:  invoke 2 4 0
3:  halt
4:  load 0 0
5:  invoke 1 7 0
6:  return
7:  load 0 0
8:  load 0 0
9:  mul
10: load 1 1
11: gt
12: ifzero 15
13: load 0 0
14: return
15: load 0 0
16: load 0 0
17: mul
18: return
```

TRIPLA → TRAM

Projekt

TRAM implementieren

Lexikalische Analyse implementieren
Parser implementieren

Berechnung der Adressumgebung,
Datenflussanalyse und
Erzeugung von TRAM-Code